In [ ]:
```
import marimo as mo
```

Verificação Formal de SFOTS com Z3

Este notebook modela e verifica formalmente um programa imperativo de multiplicação de dois números a e b, armazenando o resultado em z. Foram utilizadas as funções do solver **Z3** para desenvolver a solução.

Uso de LLMs

A LLM foi usada em diversas etapas deste trabalho, como:

- Obter uma melhor compreensão do enunciado
- Explicar conceitos de lógica computacional e SFOTS
- Definir a função `k_inducao` para provar propriedades de invariança, segurança e terminação
- Alterar trechos de código que estavam causando problemas para provar as propriedades desejadas
- Passar os comentários para a sintaxe do Marimo

Links das conversas:

Gemini Share

https://share.gemini.google/lYnlK8iHUL9Q

(Nota: Também há um arquivo JSON com a conversa com o Copilot, cujo link não foi possível compartilhar diretamente).

In [ ]:
```
from z3 import (
    BitVecs, Ints, And, Or, Not, Implies, Solver, unsat, sat,
    ULE, UGT, UGE, ULT, LShR, BVAddNoOverflow, BVMulNoOverflow
)
```

ParâmetrosGlobais

In [ ]:
```
n = 4
```

Alínea 1: Modelação do SFOTS

- As variáveis do modelo são vetores de bits de tamanho n.

- O estado inicial é definido por conjunções de várias condições, enquanto os demais estados não são definidos diretamente, mas sim por um ponteiro (`loc`) que navega entre estados.
- A transição entre estados é definida por conjunções de várias condições, que representam as regras de transição do programa.
- O estado de erro é acionado quando há overflow na soma ou no shift, e o programa termina quando y = 0.

In [ ]:
```
a, b, x, y, z = BitVecs('a b x y z', n)
x_next, y_next, z_next = BitVecs('x_next y_next z_next', n)
a_next, b_next = BitVecs('a_next b_next', n)

loc, loc_next = Ints('loc loc_next')

inputs = And(a_next == a, b_next == b)

init = And(
    loc == 0,
    x == a,
    y == b,
    z == 0,
    a >= 0,
    b >= 0,
)

overflow_soma = Or(Not(BVAddNoOverflow(z, x, False)))
overflow_shift = UGE(x, 1 << (n - 1))

tr_0_1 = And(loc == 0, loc_next == 1, x_next == x, y_next == y, z_next == z)
tr_loop_end = And(
    loc == 1, y == 0, loc_next == 2, x_next == x, y_next == y, z_next == z
)
tr_erro = And(
    loc == 1,
    y != 0,
    loc_next == 3,
    Or(
        And((y & 1) == 0, overflow_shift),
        And((y & 1) != 0, overflow_soma),
    )
)
tr_par = And(
    loc == 1,
    y != 0,
    (y & 1) == 0,
    Not(overflow_shift),
    loc_next == 1,
    x_next == x << 1,
    y_next == LShR(y, 1),
    z_next == z,
)
tr_impar = And(
    loc == 1,
```

```
    y != 0,
    (y & 1) != 0,
    Not(overflow_soma),
    loc_next == 1,
    x_next == x,
    y_next == y - 1,
    z_next == z + x,
)

transicao = And(
    Or(tr_0_1, tr_loop_end, tr_erro, tr_par, tr_impar), inputs
)
```

In [ ]:
```
def k_inducao(nome, P, P_next, transicao, custom_init):
    s_base = Solver()
    s_base.add(custom_init)
    s_base.add(Not(P))
    res_base = s_base.check()

    s_ind = Solver()
    s_ind.add(P)
    s_ind.add(transicao)
    s_ind.add(Not(P_next))
    res_ind = s_ind.check()

    output = []
    output.append(f"### [{nome}]")
    output.append(f"- **Caso Base:** {'Válido (unsat)' if res_base == unsat else
'Falhou'}")
    output.append(f"- **Passo Indutivo:** {'Válido (unsat)' if res_ind == unsat else
'Falhou'}")
    if res_base == unsat and res_ind == unsat:
        output.append("-> **Resultado:** PROVADO por k-indução!\n")
    else:
        output.append("-> **Resultado:** Não provado diretamente.\n")
    return "\n".join(output)
```

Alínea 2: Prova do Invariante

Para provar a propriedade $(x \cdot y + z = a \cdot b)$ é válida neste SFOTS, foi usada a técnica de k-indução, provando a validade de uma propriedade no caso base e no passo indutivo. Para isso, com o auxílio da LLM, foi definida uma função `k_inducao` que recebe como parâmetros o nome da propriedade, a propriedade P, a propriedade P para o próximo estado e um estado de inicialização, que por padrão é o estado `init` definido no começo do programa. Dessa forma, a invariante para o caso base e para o passo indutivo foram definidas através de conjunções de implicações e então passadas para a função `k_inducao`. A resposta esperada é que a propriedade seja provada.

In [ ]:
```
inv = And(
    Implies(loc == 0, And(x == a, y == b, z == 0)),
    Implies(Or(loc == 1, loc == 2), x * y + z == a * b)
```

```
)
inv_next = And(
    Implies(loc_next == 0, And(x_next == a_next, y_next == b_next, z_next == 0)),
    Implies(Or(loc_next == 1, loc_next == 2), x_next * y_next + z_next == a_next *
b_next)
)

res_inv = k_inducao('Invariante (x*y + z == a*b)', inv, inv_next, transicao, init)
mo.md(res_inv)
```

[Invariante (*xy* + *z* == *ab*)]

- **Caso Base:** Válido (unsat)
- **Passo Indutivo:** Válido (unsat) -> **Resultado:** PROVADO por k-indução!

Alínea 3: Prova de Segurança

Para provar a propriedade de segurança $(a, b, x, y, z) \in [0, 2^{n/2} - 1]$, foi novamente usada a técnica de k-indução, mas dessa vez com uma série de restrições de segurança para as variáveis e para o estado inicial, que foram definidos através de recursos do Z3 como `ULE`, `UGT`, `Implies`, `BVMulNoOverflow` e `BVAddNoOverflow`. Após isso, a função `k_inducao` foi chamada com essas restrições e o estado inicial seguro, e a resposta esperada é que a propriedade seja provada.

In [ ]:
```
limite = 2 ** (n // 2)
init_seguro = And(init, a < limite, b < limite)

limites_estado = And(
    ULE(y, b),
    ULE(z, a * b),
    ULE(x, a * (b + 1)),
    Implies(UGT(y, 0), ULE(x, a * b)),
    BVMulNoOverflow(x, y, False),
    BVAddNoOverflow(x * y, z, False),
)
limites_estado_next = And(
    ULE(y_next, b_next),
    ULE(z_next, a_next * b_next),
    ULE(x_next, a_next * (b_next + 1)),
    Implies(UGT(y_next, 0), ULE(x_next, a_next * b_next)),
    BVMulNoOverflow(x_next, y_next, False),
    BVAddNoOverflow(x_next * y_next, z_next, False),
)

seguranca = And(
    inv,
    loc != 3,
    ULE(a, limite - 1),
    ULE(b, limite - 1),
    limites_estado,
```

```
)
seguranca_next = And(
    inv_next,
    loc_next != 3,
    ULE(a_next, limite - 1),
    ULE(b_next, limite - 1),
    limites_estado_next,
)

res_seg = k_inducao(
    'Segurança (Sem Overflow)',
    seguranca,
    seguranca_next,
    transicao,
    custom_init=init_seguro
)
mo.md(res_seg)
```

[Segurança (Sem Overflow)]

- **Caso Base:** Válido (unsat)
- **Passo Indutivo:** Válido (unsat) -> **Resultado:** PROVADO por k-indução!

Alínea 4: Prova de Terminação (Animação)

Para provar a propriedade de terminação (y decresce ou zera), foi usada a técnica de k-indução uma última vez, definindo uma propriedade de progresso que representa a condição de terminação do programa. Em seguida, usamos o solver para verificar que ao haver uma transição, o trecho `Not(progresso)` não é satisfazível, logo, a cada iteração, y sempre é decrementado ou zera, e o programa termina, quaisquer que sejam os valores de a e b iniciais. A resposta esperada é que a propriedade seja provada.

In [ ]:
```
progresso = Or(ULT(y_next, y), y == 0)

s_term = Solver()
s_term.add(
    loc == 1,
    UGT(y, 0),
    transicao,
    loc_next != 3,
    Not(progresso),
)
res_term = s_term.check()

output_term = []
output_term.append("### [Terminação]")
output_term.append(f"- **Verificação de Progresso (y decresce ou zera):** {'Válido
(unsat)' if res_term == unsat else 'Falhou'}")
if res_term == unsat:
    output_term.append("-> **Provado:** O programa termina sempre.\n")
```

```
mo.md("\n".join(output_term))
```

[Terminação]

- **Verificação de Progresso (y decresce ou zera):** Válido (unsat) -> **Provado:** O programa termina sempre.